Word Vectors-I

(Introduction,Word2Vec, Skipgram Model)

DR. JASMEET SINGH,
ASSISTANT PROFESSOR,
CSED, TIET

Word meaning representation

- Definition: **meaning** (Webster dictionary)
  - the idea that is represented by a word, phrase, etc.
  - the idea that a person wants to express by using words, signs, etc.
  - the idea that is represented in a work of writing, art, etc.

- **Commonest linguistic way of thinking of meaning:**

  signifier (symbol) ↔ signified (idea or thing)

How do we have usable meaning in Computer?

Common NLP solution: Use, e.g., WordNet, a thesaurus containing lists of **synonym sets** and **hypernyms** ("is a" relationships).

e.g., synonym sets containing "good":

```
from nltk.corpus import wordnet as wn
poses = { 'n':'noun', 'v':'verb', 's':'adj (s)', 'a':'adj', 'r':'adv'}
for synset in wn.synsets("good"):
    print("{}: {}".format(poses[synset.pos()],
            ", ".join([l.name() for l in synset.lemmas()])))
```

```
noun: good
noun: good, goodness
noun: good, goodness
noun: commodity, trade_good, good
adj: good
adj (sat): full, good
adj: good
adj (sat): estimable, good, honorable, respectable
adj (sat): beneficial, good
adj (sat): good
adj (sat): good, just, upright
…
adverb: well, good
adverb: thoroughly, soundly, good
```

e.g., hypernyms of "panda":

```
from nltk.corpus import wordnet as wn
panda = wn.synset("panda.n.01")
hyper = lambda s: s.hypernyms()
list(panda.closure(hyper))
```

```
[Synset('procyonid.n.01'),
Synset('carnivore.n.01'),
Synset('placental.n.01'),
Synset('mammal.n.01'),
Synset('vertebrate.n.01'),
Synset('chordate.n.01'),
Synset('animal.n.01'),
Synset('organism.n.01'),
Synset('living_thing.n.01'),
Synset('whole.n.02'),
Synset('object.n.01'),
Synset('physical_entity.n.01'),
Synset('entity.n.01')]
```

Problems with WordNet

- Great as a resource but missing nuance
  - e.g., “proficient” is listed as a synonym for “good”; which is not correct in all contexts.
- Missing new meanings of words; impossible to keep up-to-date !
- Subjective
- Requires human labor to create and adapt
- Can’t compute accurate word similarity.

Representing words as discrete symbols

- In traditional NLP, we regard words as discrete symbols: *banking*, *hotel*, *conference*, *motel* –often referred as a **localist representation** in deep learning.
- In statistical machine learning, these symbols are represented using Count Vectorizers, tf-idf vectors, one-hot encoding, etc.

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]
hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

- Vector dimension= number of words in vocabulary (e.g., 250,000)

Problems with words as discrete symbols

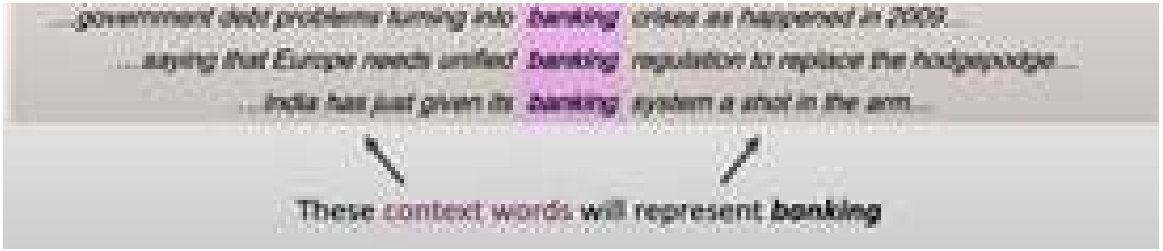
- The major problem is **no natural notion of similarity or word relationships** between these count/tf-idf/one-hot vectors.
  - For example, in case of web search, if user searches "Patiala motel", we would like to match documents containing "Patiala hotel"
  - But, the following vectors are orthogonal and does not depict any relationship

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]
hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

- **Solutions**
- Could try to rely on WordNet's list of synonyms to get similarity ?
  - But, it is well-known to fail badly; incompleteness.
- **Learn to encode similarity in these vectors themselves.**

Representing words by their contexts

- **Distributional Semantics**: A word's meaning is given by the words that frequently appear close to it.
  - "*You shall know a word by the company it keeps.*" (J.R. Firth 1957)
  - One of the most successful ideas of modern NLP.
- When a word *w* appears in text, its context is the set of words that appear nearby (within a fixed-size window).
- Use the many contexts of w to build representation of w.

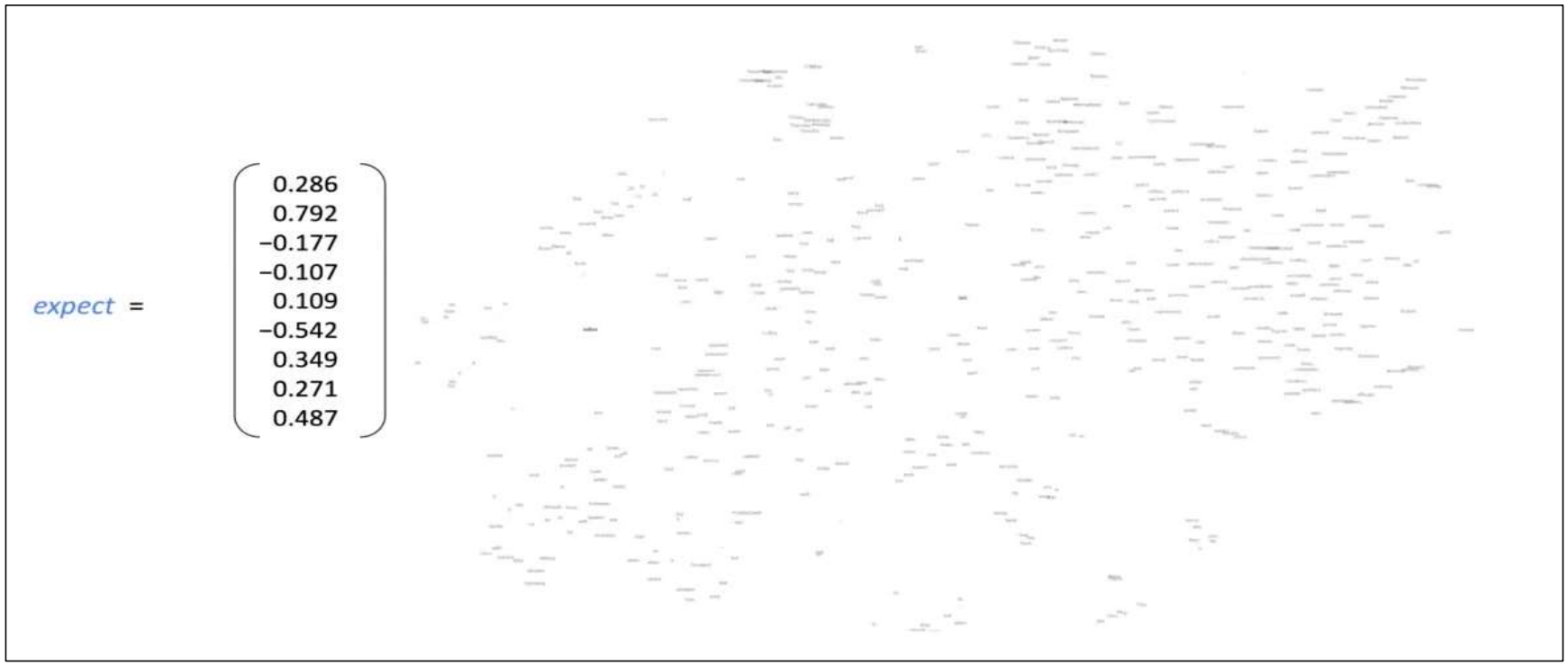


Word Vectors

- Word vector is a dense real-valued vector representation for a word
  - build by looking at the words with which it appears
  - and in some sense represents the meaning of the word.
- In other words, it is similar to vectors of words that appear in similar contexts.
- Word vectors are also called as *word embeddings* or *(neural) word representations*.
- These are distributed representations and not just localist representation.

$$\textit{banking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$$

Word Vectors Visualization



Word Vectors Visualization (Contd…)

expect =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$

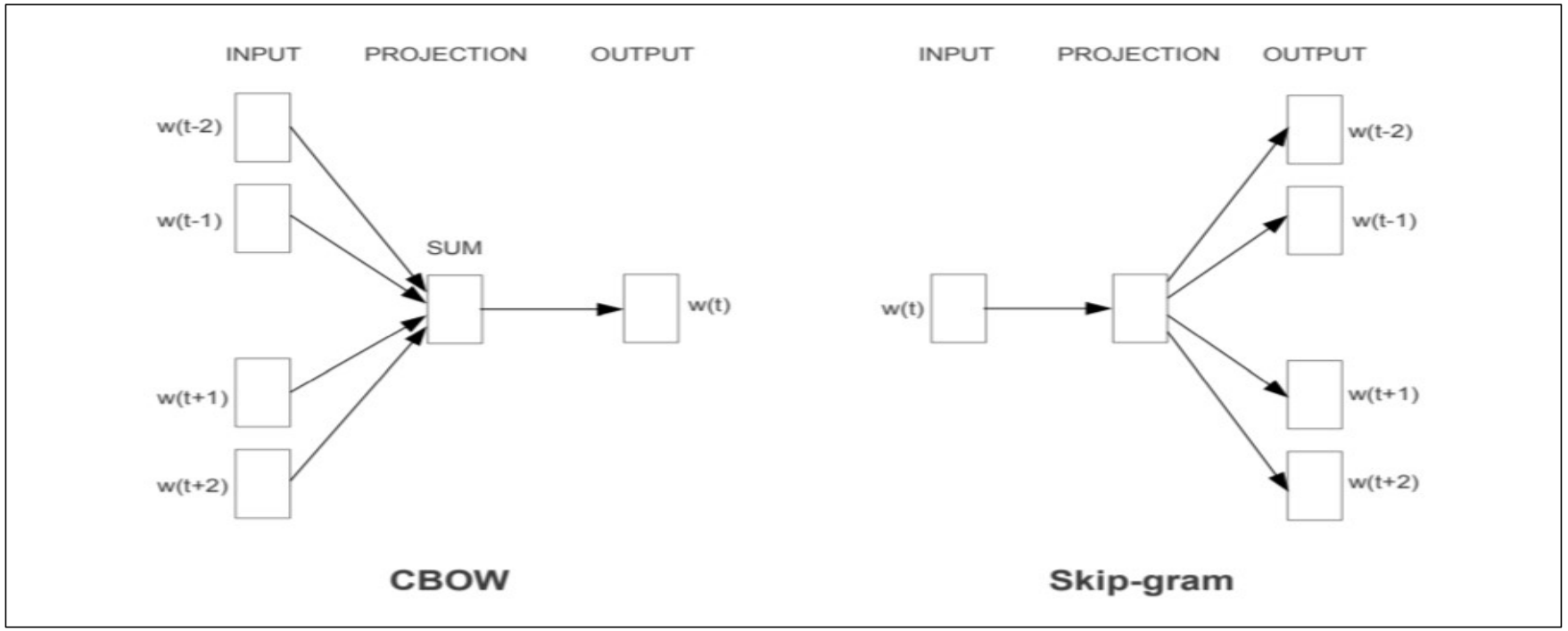
Word Vectors Visualization (Contd…)

$$expect = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$

Word2Vec

- Word2Vec is a framework for learning word vectors proposed by Mikolov et al. (2013).
- The original Word2Vec paper proposed two neural network architectures: *Continuous Bag-of-Words* (CBOW) and *Skip-gram*.
- Both are architectures to learn the underlying word representations for each word by using neural networks.
- In the **CBOW** model, the distributed representations of context (or surrounding words) are combined to **predict the word in the middle**. While in the **Skip-gram** model, the distributed representation of the input word is used to **predict the context**.

Word2Vec (Contd…)

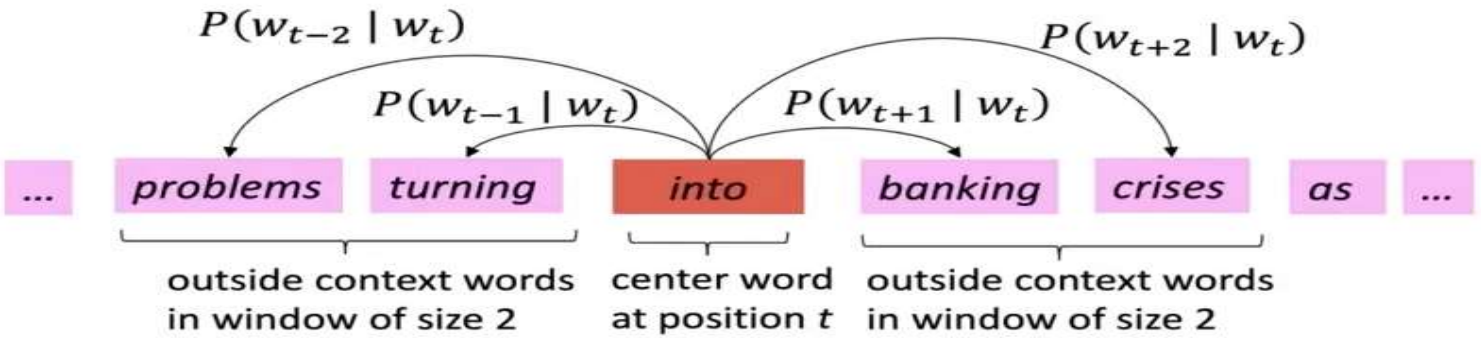


Skip-gram Model

- The basic idea of skip-gram model is as follows:
  - We have a large corpus (“body”) of text.
  - Each word in a fixed vocabulary is represented by a vector.
  - Go through each position t in the text, which has center word c and context words o (“outside”)
  - Use the similarity between the word vectors of c and o to calculate the probability of o given c.
  - Keep adjusting the word vectors to maximize this probability.

Skip-gram Overview

Example windows and process for computing $P(w_{t+j} \mid w_t)$



Skip-gram: Objective Function

- For each position, t= 1,2,….., T predict context words within a window of size m, given a center word w_t and compute data likelihood:

$$Likelihood = L(\theta) = \prod_{t=1}^{T} \prod_{\substack{-m \le j \le m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$$

Where θ are the set of variables that needs to be minimized.

- The objective function J(θ) for the skip-gram model is the average negative log of the likelihood (because minimizing objective function is same as maximizing probability).

$$J(\theta) = -\frac{1}{T}\log(L(\theta)) = -\frac{1}{T} log \left(\prod_{t=1}^{T} \prod_{\substack{-m \le j \le m \\ j \neq 0}} P(w_{t+j} | w_t; \theta) \right) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-m \le j \le m \\ j \neq 0}} \log(P(w_{t+j} | w_t; \theta))$$

Skip-gram: Objective Function (Contd….)

- How to compute $P(w_{t+j}|w_t;\theta)$ for the objective function?
  - For this, we use two vectors for each word w:
    - v_w *when the word is a center word*
    - u_w *when the word is a context* $(outside)$*word.*
  - Then, probability of context word (o, w_{t+j}) given a center word (c, w_t) is computed as follows:

$$P(o|c) = \frac{\exp(u_o^T . v_c)}{\sum_{w \epsilon V} \exp(u_w^T . v_c)}$$

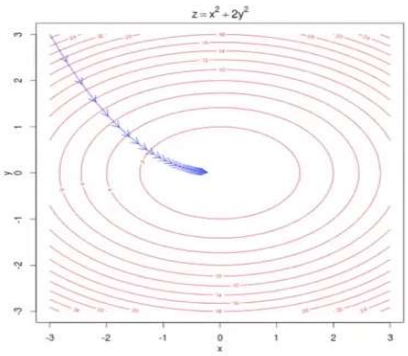
  - The probability function is a softmax function that amplifies the probability with largest dot product in the numerator (max) and still provides some probability to smallest dot product (soft).

Skip-gram Model: Training

- To train the skip-gram model, we gradually adjust the parameters to minimize the loss using mini-batch gradient descent.
- In this case, θ (the model parameters) is one long vector containing 2V vectors (V is vocabulary and each word in vocabulary has 2 vectors) and each of this 2V vectors is of d dimensions. (see fig a)
- We optimize these parameters by walking down the gradients (as shown in fig b).
- Hence, we compute all vector gradients.

$$\theta = \begin{bmatrix} v_{aardvark} \\ v_a \\ \vdots \\ v_{zebra} \\ u_{aardvark} \\ u_a \\ \vdots \\ u_{zebra} \end{bmatrix} \in \mathbb{R}^{2dV}$$

(a)



(b)

Skip-gram Model: Computing Gradients

- As discussed,

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{-m\le j\le m \\ j\neq 0}} \log\left(P\left(w_{t+j}\middle|w_t;\theta\right)\right)$$

$$P(o|c) = \frac{\exp(u_o^T.v_c)}{\sum_{w\epsilon} \exp(u_w^T.v_c)}$$

Therefore, $\log\left(P(o|c)\right) = log\frac{\exp(u_o^T.v_c)}{\sum_{w\epsilon V}\exp(u_w^T.v_c)} = \log(\exp(u_o^T.v_c)) - log\sum_{w\epsilon V}\exp(u_w^T.v_c)$

$= u_o^T.v_c - log\sum_{w\epsilon V}\exp(u_w^T.v_c)$

$\frac{\partial j(\theta)}{\partial v_c} = -\frac{\partial(u_o^T.v_c - log\sum_{w\epsilon V}\exp(u_w^T.v_c))}{\partial v_c} = -\frac{\partial(u_o^T.v_c)}{\partial v_c} + \frac{\partial log\sum_{w\epsilon V}\exp(u_w^T.v_c)}{\partial v_c} = -Term1 + Term2$

Skip-gram Model: Computing Gradients (Contd……)

$$Term1 = \frac{\partial(u_o^T . v_c)}{\partial v_c} = u_o$$

$$Term2 = \frac{\partial log \sum_{w \epsilon V} \exp(u_w^T . v_c)}{\partial v_c} = \frac{1}{\sum_{w=1}^{V} \exp(u_w^T . v_c)} \frac{\partial \sum_{x=1}^{V} \exp(u_x^T . v_c)}{\partial v_c}$$

$$= \frac{1}{\sum_{w=1}^{V} \exp(u_w^T . v_c)} \sum_{x=1}^{V} \frac{\partial(\exp(u_x^T . v_c))}{\partial v_c} = \frac{1}{\sum_{w=1}^{V} \exp(u_w^T . v_c)} \sum_{x=1}^{V} \exp(u_x^T . v_c) \frac{\partial(u_x^T . v_c)}{\partial v_c}$$

$$= \frac{1}{\sum_{w=1}^{V} \exp(u_w^T . v_c)} \sum_{x=1}^{V} \exp(u_x^T . v_c) u_x = \sum_{x=1}^{V} P(x|c) u_x$$

$$Therefore, \frac{\partial j(\theta)}{\partial v_c} = -u_o + \sum_{x=1}^{V} P(x|c) u_x$$

Skip-gram Model: Computing Gradients (Contd….)

There is something very interesting about the equation just derived . The first part is the current representation of the context word.

The second part is an expectation of what the context word should look like according to our model as we are taking the current representation of all context words and multiplying them with their probability in the current model.

So basically, we are subtracting the actual and expected representations to get the direction in which I should move and change my weight vector ***Vc*** so that I can maximize the likelihood.

Skip-gram Model: Computing Gradients (Contd….)

Moving forward in the same manner, we can also calculate the derivative of $\boldsymbol{J(\theta)}$ wrt to $\boldsymbol{u_w}$. There will be two cases for $\boldsymbol{u_w}$, one when w is the context word and one when w is not the context word.

Case I: When $w \neq o$

Then $\frac{\partial j(\theta)}{\partial u_w} = P(x|c)v_c$

Case II: When $w = o$

Then $\frac{\partial j(\theta)}{\partial u_o} = -v_c + P(o|c)v_c$

Skip-gram Model: Summary (Pseudocode)

1. Extract set of unique words i.e. vocabulary (V) from the long body of text.
2. Randomly initialize two vectors ($\vec{u}$ and $\vec{v}$) for each word $w \epsilon V$.
3. For i =1 to n iterations:
4. For t = 1 to T position in text:
5. For each outside word o in the fixed window
6. Compute $\frac{\partial J}{\partial v_c} = -u_o + \sum_{x=1}^{w} P(x|c) * u_x$,

$\frac{\eth J}{\partial u_o} = -v_c + P(o|c)v_c$

$\frac{\partial J}{\partial u_w} = P(x|c) * v_c$

6. Update: $v_c = v_c - \alpha \frac{\partial J}{\partial v_c}, u_o = u_o - \alpha \frac{\eth J}{\partial u_o}, u_w = u_w - \alpha \frac{\partial J}{\partial u_w}$
7. Average the $\vec{u}$ and $\vec{v}$ of each word w.